Red Neuronal

Nombre: Erick Mejia

NUESTRA PRIMERA RED NEURONAL: MNIST DATASET

In [ ]:
```
!pip install $packageName
```

In [ ]:
```
!pip freeze
```

Hola voy a ejecutar el comando `!pip freeze` para cotillear a Google Colab

In [ ]:
```
!nvidia-smi
```

In [1]:
```
#Importemos TensorFlow 2.X y Numpy
import numpy as np
import tensorflow as tf
tf.__version__
```

Out[1]:
```
'2.21.0'
```

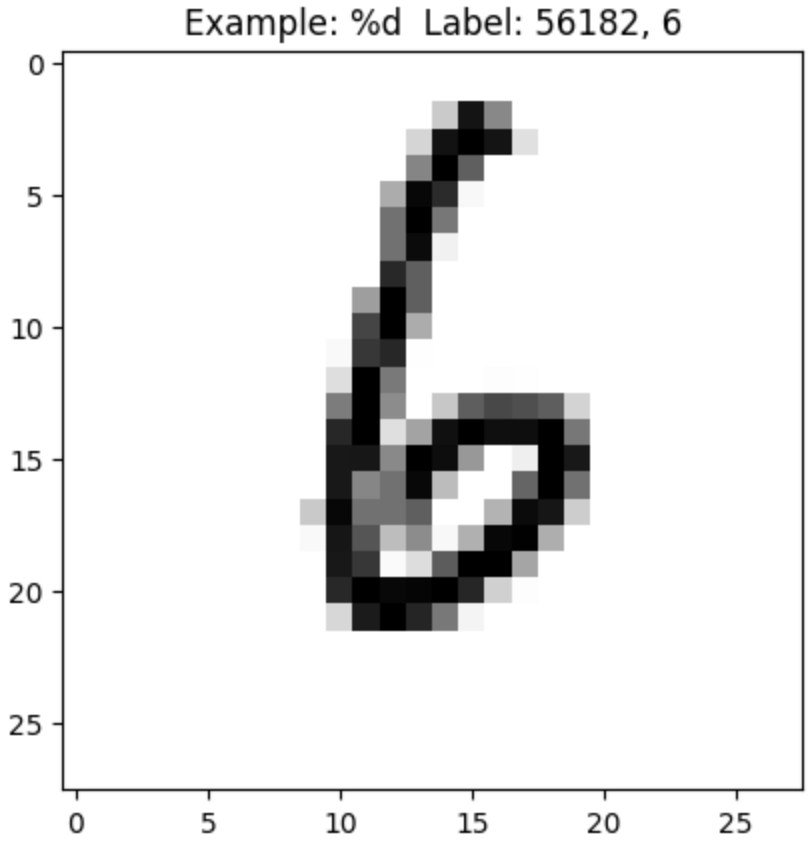
- Cargando el conjunto de datos

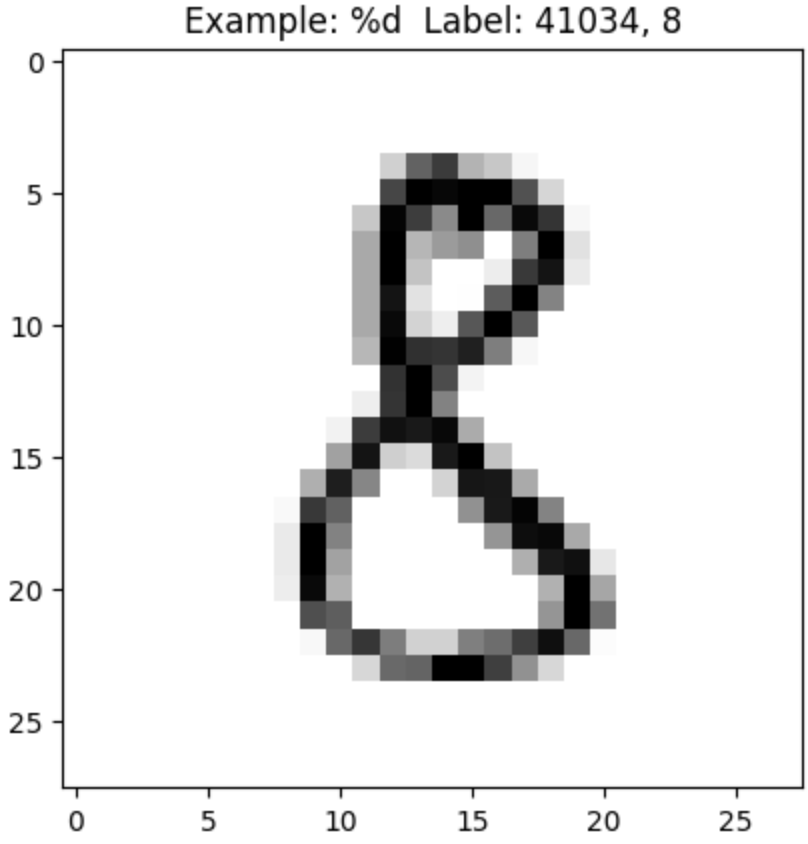
In [2]:
```
# Importamos el dataset MNIST y cargamos los datos
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
print(x_train.shape)
print(y_train.shape)
print(x_test.shape)
print(y_test.shape)
```

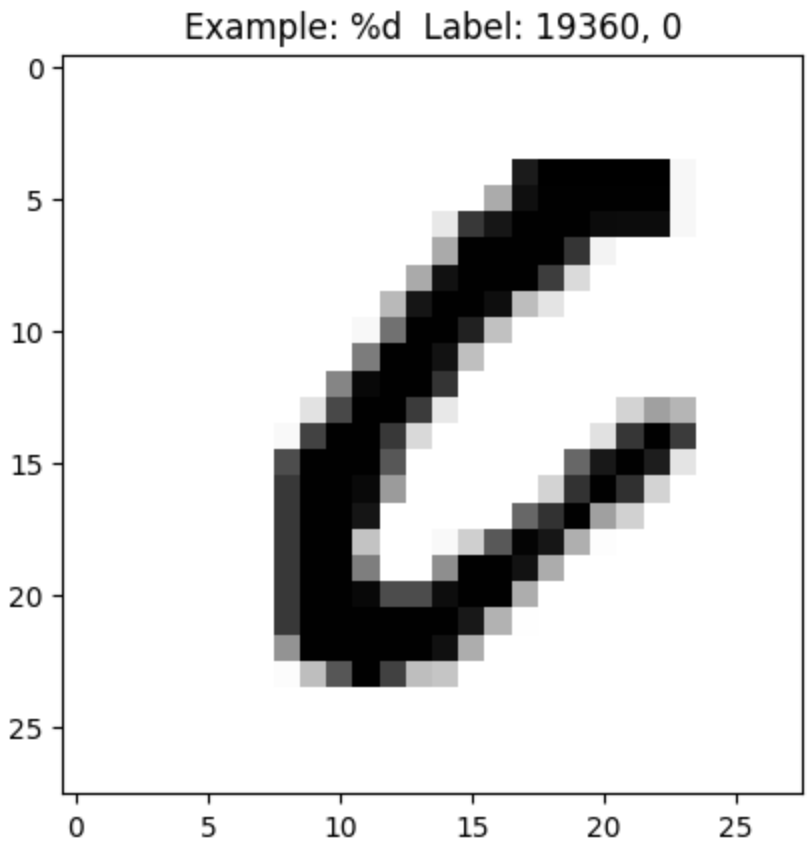
```
(60000, 28, 28)
(60000,)
(10000, 28, 28)
(10000,)
```

- Inspeccionando el conjunto de datos

In [3]:
```
import matplotlib.pyplot as plt
# Función auxiliar para visualizar datos de entrenamiento de manera aleatoria
def display_digit(num,x_train,y_train):
  # Seleccionar la imagen num de mnist.train.images y hacer un reshape al tamaño de
  image = x_train[num,:,:]
  # Seleccionar el target num de mnist.train.labels
  label = y_train[num]
  # Mostrar
  plt.title(f'Example: %d  Label: {num}, {label}')
  plt.imshow(image, cmap=plt.get_cmap('gray_r'))
  plt.show()

# Mostramos algunos ejemplos
display_digit(np.random.randint(0, x_train.shape[0]),x_train,y_train)
display_digit(np.random.randint(0, x_train.shape[0]),x_train,y_train)
display_digit(np.random.randint(0, x_train.shape[0]),x_train,y_train)
```


Example: %d Label: 56182, 6
0
5
10
15
20
25
0
5
10
15
20
25


Example: %d Label: 41034, 8
0
5
10
15
20
25
0
5
10
15
20
25



In [4]:
```
np.random.randint(0, x_train.shape[0])
```

Out[4]: 12680

In [ ]:

- Acondicionando el conjunto de datos

In [5]:
```
# Pre-procesado obligatorio cuando trabajo con redes neuronales
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten
from sklearn.model_selection import train_test_split

x_train, x_te = x_train / 255.0, x_test / 255.0 #Cambio al rango 0-1 -> Disminuyo C
y_train = to_categorical(y_train, num_classes=10) #One-hot encoding para minimizar
print(y_train.shape)
y_te = to_categorical(y_test, num_classes=10)
x_tr, x_val, y_tr, y_val = train_test_split(x_train, y_train, test_size=0.1, random
print(y_tr.shape)
# print(y_tr[0])
```

```
(60000, 10)
(54000, 10)
```

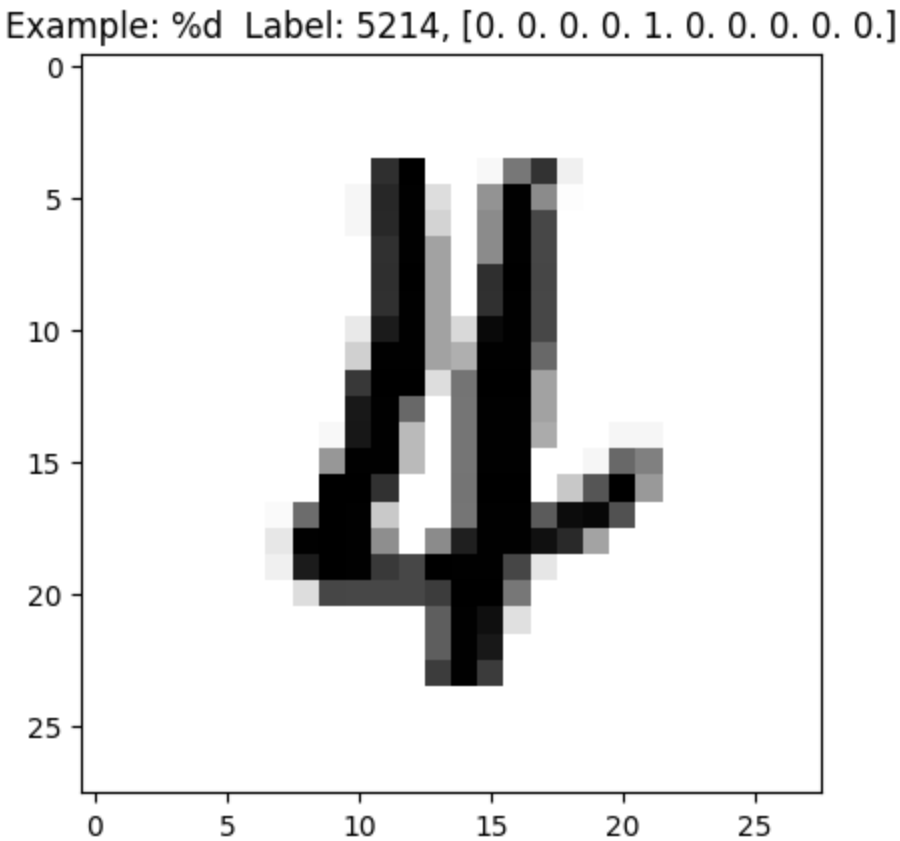
In [6]:
```
print(y_tr[5214])
```

```
[0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
```

In [7]:
```
print(y_tr[5214].argmax(axis=0))
```

4

In [8]:
```
display_digit(5214, x_tr,y_tr)
```



- Creando la topología de Red Neuronal (MLP) y entrenándola

In [9]:
```
# Voy a necesitar importar una serie de modulos para programar mi red neuronal
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Input, Dropout
# Vamos a codificar la topología de nuestra primera red neuronal!!!
model = Sequential()
model.add(Flatten(input_shape=(28,28)))
model.add(Dense(512, input_shape=(28*28,), activation="relu"))
model.add(Dense(10, activation="softmax")) #Capa salida -> Mismo número de neuronas

# Ahí tenemos nuestro primer MLP con una única capa oculta de 512 neuronas
```

```
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\keras\src\layers\reshaping
\flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
layer. When using Sequential models, prefer using an `Input(shape)` object as the fi
rst layer in the model instead.
  super().__init__(**kwargs)
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\keras\src\layers\core\dens
e.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
When using Sequential models, prefer using an `Input(shape)` object as the first lay
er in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [10]:
```
#Traigamos nuestro optimizador
from tensorflow.keras.optimizers import SGD
# Ahora que tengo definida la arquitectura, la compilo
model.compile(loss="categorical_crossentropy", optimizer=SGD(0.005), metrics=["accu
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
```

In [11]:
```
######### Modelo funcional ############ (Clase 04/10/2023)
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Dropout
from tensorflow.keras.optimizers import SGD

# Definir la entrada
input_layer = Input(shape=(28, 28))

# Capa de aplanamiento
x = Flatten()(input_layer)
x = Dense(512, activation="relu")(x)
# x = Dropout(0.5)(x)  # Introducción de capa de dropout en un modelo funcional
output_layer = Dense(10, activation="softmax")(x)

# Crear el modelo
model = Model(inputs=input_layer, outputs=output_layer)

# Compilar el modelo
model.compile(
    loss="categorical_crossentropy",
    optimizer=SGD(learning_rate=0.005),
    metrics=["accuracy"]
)

# Guardamos este modelo para usarlo después en la predicción de imágenes
model_mnist = model
```

In [12]:
```
print(model.summary())
```

Model: "functional_3"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 28, 28)	0
flatten_1 (Flatten)	(None, 784)	0
dense_2 (Dense)	(None, 512)	401,920
dense_3 (Dense)	(None, 10)	5,130

Total params: 407,050 (1.55 MB)

Trainable params: 407,050 (1.55 MB)

Non-trainable params: 0 (0.00 B)

```
None
```

In [13]:

```
# Por fin podemos entrenar nuestra primera red neuronal
print("[INFO]: Entrenando red neuronal...")
H = model.fit(x_tr, y_tr, validation_data=(x_val, y_val), epochs=50, batch_size=128
model_mnist = model
```

```
[INFO]: Entrenando red neuronal...
Epoch 1/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 10s 18ms/step - accuracy: 0.6708 - loss: 1.4720 - val_a
ccuracy: 0.8095 - val_loss: 0.9760
Epoch 2/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.8347 - loss: 0.7795 - val_ac
curacy: 0.8530 - val_loss: 0.6575
Epoch 3/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.8612 - loss: 0.5862 - val_a
ccuracy: 0.8682 - val_loss: 0.5372
Epoch 4/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.8751 - loss: 0.5009 - val_ac
curacy: 0.8822 - val_loss: 0.4742
Epoch 5/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 11s 16ms/step - accuracy: 0.8838 - loss: 0.4519 - val_a
ccuracy: 0.8900 - val_loss: 0.4342
Epoch 6/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 17ms/step - accuracy: 0.8906 - loss: 0.4194 - val_ac
curacy: 0.8943 - val_loss: 0.4065
Epoch 7/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.8951 - loss: 0.3958 - val_ac
curacy: 0.8975 - val_loss: 0.3851
Epoch 8/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 17ms/step - accuracy: 0.8988 - loss: 0.3776 - val_ac
curacy: 0.9013 - val_loss: 0.3688
Epoch 9/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9021 - loss: 0.3629 - val_ac
curacy: 0.9042 - val_loss: 0.3553
Epoch 10/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9043 - loss: 0.3508 - val_ac
curacy: 0.9063 - val_loss: 0.3440
Epoch 11/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9068 - loss: 0.3404 - val_ac
curacy: 0.9093 - val_loss: 0.3339
Epoch 12/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9088 - loss: 0.3312 - val_ac
curacy: 0.9112 - val_loss: 0.3252
Epoch 13/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9106 - loss: 0.3233 - val_ac
curacy: 0.9120 - val_loss: 0.3178
Epoch 14/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9124 - loss: 0.3160 - val_ac
curacy: 0.9137 - val_loss: 0.3110
Epoch 15/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9141 - loss: 0.3094 - val_ac
curacy: 0.9155 - val_loss: 0.3046
Epoch 16/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9155 - loss: 0.3033 - val_ac
curacy: 0.9160 - val_loss: 0.2987
Epoch 17/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9175 - loss: 0.2977 - val_ac
curacy: 0.9173 - val_loss: 0.2933
Epoch 18/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9188 - loss: 0.2924 - val_ac
curacy: 0.9197 - val_loss: 0.2888
Epoch 19/50
```

```
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9201 - loss: 0.2874 - val_ac
curacy: 0.9188 - val_loss: 0.2844
Epoch 20/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 17ms/step - accuracy: 0.9217 - loss: 0.2828 - val_ac
curacy: 0.9203 - val_loss: 0.2799
Epoch 21/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 8ms/step - accuracy: 0.9230 - loss: 0.2784 - val_acc
uracy: 0.9215 - val_loss: 0.2756
Epoch 22/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 8s 16ms/step - accuracy: 0.9242 - loss: 0.2742 - val_ac
curacy: 0.9232 - val_loss: 0.2713
Epoch 23/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 10s 15ms/step - accuracy: 0.9252 - loss: 0.2701 - val_a
ccuracy: 0.9237 - val_loss: 0.2682
Epoch 24/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9262 - loss: 0.2663 - val_ac
curacy: 0.9253 - val_loss: 0.2643
Epoch 25/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 15ms/step - accuracy: 0.9273 - loss: 0.2626 - val_ac
curacy: 0.9277 - val_loss: 0.2603
Epoch 26/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9285 - loss: 0.2590 - val_ac
curacy: 0.9280 - val_loss: 0.2570
Epoch 27/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.9295 - loss: 0.2556 - val_a
ccuracy: 0.9287 - val_loss: 0.2539
Epoch 28/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 9s 12ms/step - accuracy: 0.9305 - loss: 0.2523 - val_ac
curacy: 0.9298 - val_loss: 0.2511
Epoch 29/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9311 - loss: 0.2491 - val_ac
curacy: 0.9298 - val_loss: 0.2481
Epoch 30/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9320 - loss: 0.2460 - val_ac
curacy: 0.9313 - val_loss: 0.2448
Epoch 31/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.9327 - loss: 0.2430 - val_a
ccuracy: 0.9313 - val_loss: 0.2422
Epoch 32/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 7ms/step - accuracy: 0.9338 - loss: 0.2401 - val_acc
uracy: 0.9323 - val_loss: 0.2392
Epoch 33/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 6s 15ms/step - accuracy: 0.9346 - loss: 0.2372 - val_ac
curacy: 0.9325 - val_loss: 0.2369
Epoch 34/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 17ms/step - accuracy: 0.9359 - loss: 0.2345 - val_ac
curacy: 0.9330 - val_loss: 0.2345
Epoch 35/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 11s 18ms/step - accuracy: 0.9364 - loss: 0.2318 - val_a
ccuracy: 0.9338 - val_loss: 0.2323
Epoch 36/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 9s 16ms/step - accuracy: 0.9369 - loss: 0.2292 - val_ac
curacy: 0.9345 - val_loss: 0.2294
Epoch 37/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9378 - loss: 0.2266 - val_ac
curacy: 0.9362 - val_loss: 0.2274
```

```
Epoch 38/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 11s 18ms/step - accuracy: 0.9384 - loss: 0.2242 - val_a
ccuracy: 0.9355 - val_loss: 0.2250
Epoch 39/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 6s 14ms/step - accuracy: 0.9391 - loss: 0.2217 - val_ac
curacy: 0.9372 - val_loss: 0.2225
Epoch 40/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9399 - loss: 0.2193 - val_ac
curacy: 0.9382 - val_loss: 0.2211
Epoch 41/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 15ms/step - accuracy: 0.9402 - loss: 0.2170 - val_ac
curacy: 0.9387 - val_loss: 0.2189
Epoch 42/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 19ms/step - accuracy: 0.9408 - loss: 0.2147 - val_a
ccuracy: 0.9390 - val_loss: 0.2162
Epoch 43/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 17ms/step - accuracy: 0.9414 - loss: 0.2125 - val_ac
curacy: 0.9397 - val_loss: 0.2141
Epoch 44/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 8s 20ms/step - accuracy: 0.9424 - loss: 0.2103 - val_ac
curacy: 0.9402 - val_loss: 0.2127
Epoch 45/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 11s 21ms/step - accuracy: 0.9428 - loss: 0.2082 - val_a
ccuracy: 0.9408 - val_loss: 0.2102
Epoch 46/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 8s 19ms/step - accuracy: 0.9435 - loss: 0.2062 - val_ac
curacy: 0.9412 - val_loss: 0.2085
Epoch 47/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 9s 17ms/step - accuracy: 0.9438 - loss: 0.2041 - val_ac
curacy: 0.9423 - val_loss: 0.2072
Epoch 48/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.9446 - loss: 0.2021 - val_a
ccuracy: 0.9418 - val_loss: 0.2052
Epoch 49/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9454 - loss: 0.2002 - val_ac
curacy: 0.9423 - val_loss: 0.2035
Epoch 50/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 6s 7ms/step - accuracy: 0.9463 - loss: 0.1981 - val_acc
uracy: 0.9432 - val_loss: 0.2021
```

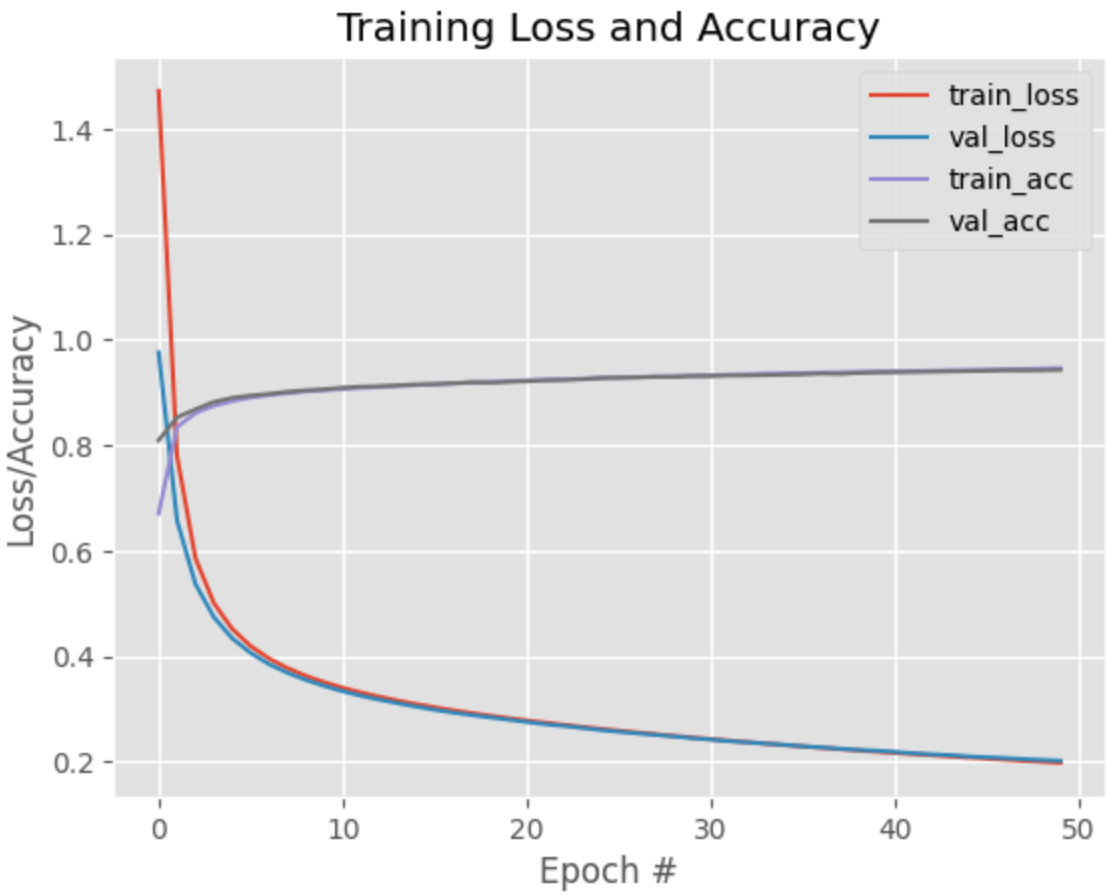
- Observando el proceso de entrenamiento para tomar decisiones

In [14]:

```
# Muestro gráfica de accuracy y losses
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 50), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 50), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 50), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 50), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
```

Out[14]:

```
<matplotlib.legend.Legend at 0x1dffc43e3d0>
```



- Probando el conjunto de datos en el subset de test y evaluando el performance del modelo

In [15]:
```
from sklearn.metrics import classification_report
# Evaluando el modelo de predicción con las imágenes de test
print("[INFO]: Evaluando red neuronal...")
predictions = model.predict(x_te, batch_size=128)
print(y_te[0])
print(predictions[0])
print(classification_report(y_te.argmax(axis=1), predictions.argmax(axis=1)))
```

```
[INFO]: Evaluando red neuronal...
79/79 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step
[0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
[6.9561516e-05 2.3900216e-07 2.9122533e-04 2.2043944e-03 1.5205635e-06
 5.5705714e-05 4.3643741e-08 9.9695146e-01 4.0680658e-05 3.8520264e-04]
              precision    recall  f1-score   support

           0       0.96      0.98      0.97       980
           1       0.98      0.98      0.98      1135
           2       0.94      0.94      0.94      1032
           3       0.92      0.94      0.93      1010
           4       0.93      0.96      0.94       982
           5       0.94      0.91      0.92       892
           6       0.94      0.96      0.95       958
           7       0.95      0.93      0.94      1028
           8       0.94      0.92      0.93       974
           9       0.94      0.92      0.93      1009

    accuracy                           0.94     10000
   macro avg       0.94      0.94      0.94     10000
weighted avg       0.94      0.94      0.94     10000
```

In [16]:
```
predictions[0].argmax(axis=0)
```

Out[16]:
```
np.int64(7)
```

In [17]:
```
y_te[0].argmax(axis=0)
```

Out[17]:
```
np.int64(7)
```

In [18]:
```
print(model.summary())
```

Model: "functional_3"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 28, 28)	0
flatten_1 (Flatten)	(None, 784)	0
dense_2 (Dense)	(None, 512)	401,920
dense_3 (Dense)	(None, 10)	5,130

Total params: 407,052 (1.55 MB)

Trainable params: 407,050 (1.55 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 2 (12.00 B)

None

MLP APLICADO A TEXTO: EJEMPLO REUTERS

- Cargando el conjunto de datos

In [19]:
```
import numpy as np
import tensorflow as tf
# Importamos el dataset REUTERS y cargamos los datos
reuters = tf.keras.datasets.reuters
WORD_LIMIT = 10000
(training_data, training_labels), (testing_data, testing_labels) = reuters.load_dat
print(training_data.shape)
print(training_labels.shape)
print(testing_data.shape)
print(testing_labels.shape)
```

```
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\numpy\lib\_format_impl.py:8
38: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy bo
olean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (De
precated NumPy 2.4)
  array = pickle.load(fp, **pickle_kwargs)
(8982,)
(8982,)
(2246,)
(2246,)
```

- Inspeccionando el conjunto de datos

In [20]:
```
# Los datos son numericos para decodificarlos, se puede usar reuters.get_word_index
word_index = reuters.get_word_index()
reverse_word_index = dict({value : key for key, value in word_index.items()})
decoded = ' '.join(
    [reverse_word_index.get(i-3,'?') for i in training_data[5]]
)
decoded
```

Out[20]:
```
"? the u s agriculture department estimated canada's 1986 87 wheat crop at 31 85 m
ln tonnes vs 31 85 mln tonnes last month it estimated 1985 86 output at 24 25 mln
tonnes vs 24 25 mln last month canadian 1986 87 coarse grain production is project
ed at 27 62 mln tonnes vs 27 62 mln tonnes last month production in 1985 86 is est
imated at 24 95 mln tonnes vs 24 95 mln last month canadian wheat exports in 1986
87 are forecast at 19 00 mln tonnes vs 18 00 mln tonnes last month exports in 1985
86 are estimated at 17 71 mln tonnes vs 17 72 mln last month reuter 3"
```

- Acondicionando el conjunto de datos

In [21]:
```
# Función auxiliar para representar las palabras (que no entiende nuestra red neuro
import numpy as np
# one hot encoding del input, vector con cada indice indicando si una palabra esta
def one_hot_encode(data):
    encoded = np.zeros((len(data),WORD_LIMIT))
    for i, v in enumerate(data):
```

```
        encoded[i,v] = 1 # localiza las columnas del genero correspondiente, marca
    return encoded
```

In [22]:
```
# Convertimos nuestras palabras a números
x_train = one_hot_encode(training_data)
x_test = one_hot_encode(testing_data)
print(x_train.shape)
print(x_test.shape)
```

```
(8982, 10000)
(2246, 10000)
```

In [23]:
```
print(x_test[3])
```

```
[0. 1. 1. ... 0. 0. 0.]
```

In [24]:
```
# Convertimos nuestros labels (categoría reseña) a one-hot encoding
from tensorflow.keras.utils import to_categorical
y_train = to_categorical(training_labels)
y_test = to_categorical(testing_labels)
print(y_train.shape)
print(y_test.shape)
```

```
(8982, 46)
(2246, 46)
```

In [25]:
```
print(y_test[3])
```

```
[0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

- Creando la topología de Red Neuronal (MLP) y entrenándola

In [26]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
# Vamos a codificar la topología de nuestro MLP
model = Sequential()
model.add(Dense(128,activation='relu',input_shape=(WORD_LIMIT,)))
model.add(Dense(64,activation='relu'))
model.add(Dense(46,activation='softmax')) # Reparto de la unidad de probabilidad en
# En el caso de regresión cambiamos la ultima capa por:
#model.add(Dense(1,activation='sigmoid'))
```

```
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\keras\src\layers\core\dens
e.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
When using Sequential models, prefer using an `Input(shape)` object as the first lay
er in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [27]:
```
# Ahora que tengo definida la arquitectura, la compilo
model.compile(optimizer='adam',
              loss='categorical_crossentropy', # ideal para clasificacion multicla
              metrics=['accuracy'])
```

In [28]:
```
# A entrenar nuestra red neuronal sea dicho!
H = model.fit(x_train,y_train,epochs=20,batch_size=32, validation_split=0.2)
```

```
Epoch 1/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 11s 33ms/step - accuracy: 0.6681 - loss: 1.5273 - val_a
ccuracy: 0.7841 - val_loss: 1.0066
Epoch 2/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 33ms/step - accuracy: 0.8761 - loss: 0.5650 - val_ac
curacy: 0.8114 - val_loss: 0.8801
Epoch 3/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 31ms/step - accuracy: 0.9410 - loss: 0.2767 - val_ac
curacy: 0.8063 - val_loss: 0.9112
Epoch 4/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.9559 - loss: 0.1917 - val_ac
curacy: 0.7930 - val_loss: 0.9651
Epoch 5/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.9557 - loss: 0.1649 - val_ac
curacy: 0.8080 - val_loss: 0.9468
Epoch 6/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 9s 25ms/step - accuracy: 0.9587 - loss: 0.1455 - val_ac
curacy: 0.7997 - val_loss: 0.9851
Epoch 7/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 6s 28ms/step - accuracy: 0.9582 - loss: 0.1337 - val_ac
curacy: 0.7969 - val_loss: 1.0428
Epoch 8/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 30ms/step - accuracy: 0.9591 - loss: 0.1213 - val_ac
curacy: 0.8002 - val_loss: 1.0151
Epoch 9/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.9603 - loss: 0.1134 - val_ac
curacy: 0.7913 - val_loss: 1.0378
Epoch 10/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - accuracy: 0.9613 - loss: 0.1063 - val_a
ccuracy: 0.7941 - val_loss: 1.0821
Epoch 11/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - accuracy: 0.9612 - loss: 0.1040 - val_a
ccuracy: 0.8036 - val_loss: 1.0779
Epoch 12/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 29ms/step - accuracy: 0.9603 - loss: 0.1005 - val_ac
curacy: 0.8008 - val_loss: 1.0830
Epoch 13/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.9616 - loss: 0.0949 - val_ac
curacy: 0.7741 - val_loss: 1.1343
Epoch 14/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 29ms/step - accuracy: 0.9637 - loss: 0.0902 - val_a
ccuracy: 0.7947 - val_loss: 1.1210
Epoch 15/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 31ms/step - accuracy: 0.9602 - loss: 0.0917 - val_ac
curacy: 0.8019 - val_loss: 1.0623
Epoch 16/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - accuracy: 0.9627 - loss: 0.0873 - val_a
ccuracy: 0.7997 - val_loss: 1.1647
Epoch 17/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.9601 - loss: 0.0862 - val_ac
curacy: 0.7952 - val_loss: 1.1631
Epoch 18/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 31ms/step - accuracy: 0.9621 - loss: 0.0837 - val_ac
curacy: 0.7902 - val_loss: 1.2018
Epoch 19/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 11s 33ms/step - accuracy: 0.9614 - loss: 0.0787 - val_a
```

```
ccuracy: 0.7974 - val_loss: 1.1751
Epoch 20/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - accuracy: 0.9620 - loss: 0.0779 - val_a
ccuracy: 0.8063 - val_loss: 1.1681
```

- Observando el proceso de entrenamiento para tomar decisiones

In [29]:
```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 128)	1,280,128
dense_5 (Dense)	(None, 64)	8,256
dense_6 (Dense)	(None, 46)	2,990

Total params: 3,874,124 (14.78 MB)

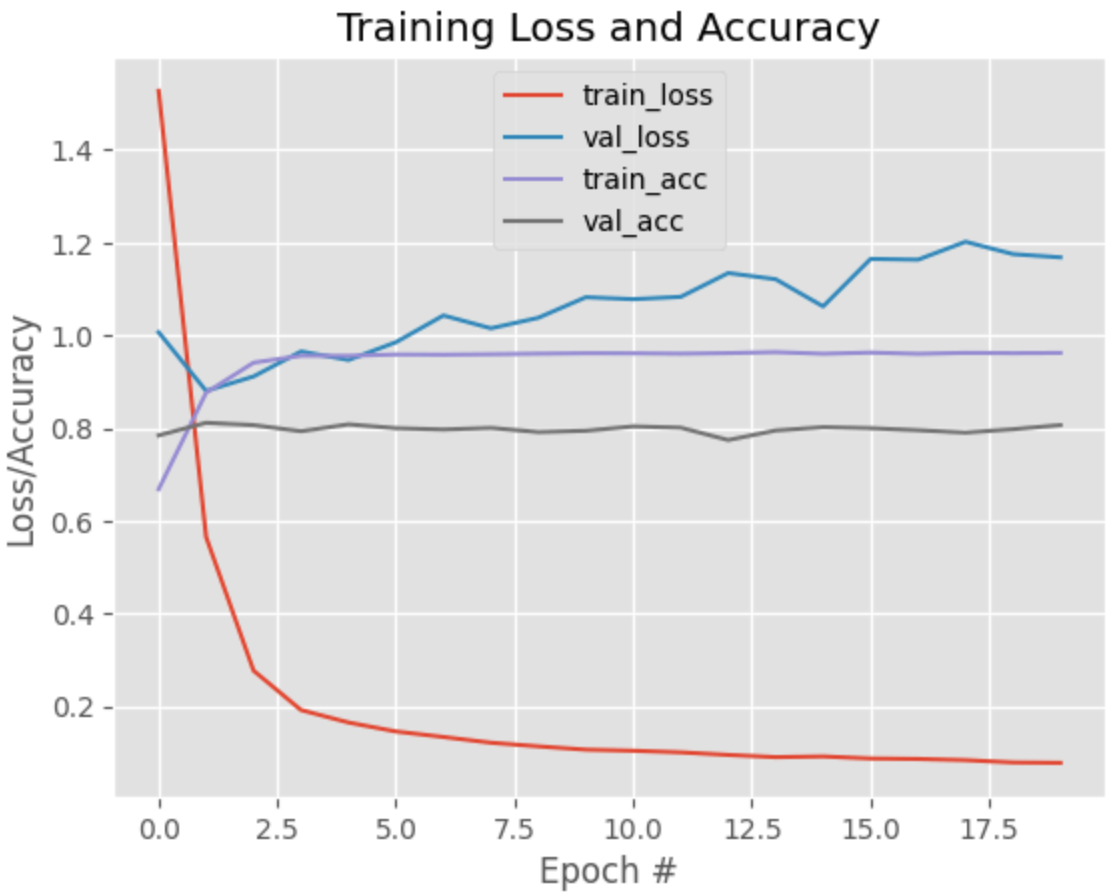
Trainable params: 1,291,374 (4.93 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 2,582,750 (9.85 MB)

In [30]:
```
import matplotlib.pyplot as plt
# Muestro gráfica de accuracy y losses
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 20), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 20), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 20), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 20), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
```

Out[30]:
```
<matplotlib.legend.Legend at 0x1dfff9cfe50>
```



- Probando el conjunto de datos en el subset de test y evaluando el performance del modelo

In [31]:
```
# Evaluando el modelo de predicción con las imágenes de test
print("[INFO]: Evaluando red neuronal...")
model.predict(x_test)
loss, accuracy = model.evaluate(x_test, y_test)
print('Loss {}, accuracy {}'.format(loss,accuracy))
```

```
[INFO]: Evaluando red neuronal...
71/71 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step
71/71 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.7916 - loss: 1.2180
Loss 1.218033790588379, accuracy 0.7916295528411865
```

REGULARIZACIÓN EN APRENDIZAJE PROFUNDO

- Weight regularization L1/L2

In [32]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras import regularizers
# Vamos a codificar la topología de nuestro MLP
model_reg = Sequential()
model_reg.add(Dense(128,activation='relu', kernel_regularizer=regularizers.l2(0.01)
```

```
model_reg.add(Dense(64,activation='relu', kernel_regularizer=regularizers.l2(0.01))
model_reg.add(Dense(46,activation='softmax')) # Reparto de la unidad de probabilida
```

```
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\keras\src\layers\core\dens
e.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
When using Sequential models, prefer using an `Input(shape)` object as the first lay
er in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [33]:
```
# Ahora que tengo definida la arquitectura, la compilo
model_reg.compile(optimizer='adam',
                  loss='categorical_crossentropy', # ideal para clasificacion multicla
                  metrics=['accuracy'])
```

In [34]:
```
# A entrenar nuestra red neuronal sea dicho!
H = model_reg.fit(x_train,y_train,epochs=20,batch_size=32, validation_split=0.2)
```

```
Epoch 1/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 13s 41ms/step - accuracy: 0.6331 - loss: 2.4536 - val_a
ccuracy: 0.7229 - val_loss: 1.8029
Epoch 2/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 9s 41ms/step - accuracy: 0.7308 - loss: 1.7236 - val_ac
curacy: 0.7440 - val_loss: 1.6784
Epoch 3/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 9s 40ms/step - accuracy: 0.7496 - loss: 1.6067 - val_ac
curacy: 0.7418 - val_loss: 1.6814
Epoch 4/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 9s 38ms/step - accuracy: 0.7598 - loss: 1.5613 - val_ac
curacy: 0.7462 - val_loss: 1.6525
Epoch 5/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 9s 39ms/step - accuracy: 0.7712 - loss: 1.5324 - val_ac
curacy: 0.7485 - val_loss: 1.6211
Epoch 6/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 9s 39ms/step - accuracy: 0.7744 - loss: 1.4823 - val_ac
curacy: 0.7546 - val_loss: 1.5900
Epoch 7/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 36ms/step - accuracy: 0.7768 - loss: 1.4681 - val_ac
curacy: 0.7518 - val_loss: 1.5812
Epoch 8/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 36ms/step - accuracy: 0.7854 - loss: 1.4273 - val_a
ccuracy: 0.7646 - val_loss: 1.5261
Epoch 9/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 9s 38ms/step - accuracy: 0.7908 - loss: 1.3997 - val_ac
curacy: 0.7668 - val_loss: 1.5706
Epoch 10/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 37ms/step - accuracy: 0.7989 - loss: 1.3733 - val_ac
curacy: 0.7618 - val_loss: 1.5250
Epoch 11/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 9s 38ms/step - accuracy: 0.8000 - loss: 1.3544 - val_ac
curacy: 0.7629 - val_loss: 1.5019
Epoch 12/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 11s 42ms/step - accuracy: 0.8010 - loss: 1.3332 - val_a
ccuracy: 0.7429 - val_loss: 1.5456
Epoch 13/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 9s 38ms/step - accuracy: 0.8089 - loss: 1.3049 - val_ac
curacy: 0.7624 - val_loss: 1.5097
Epoch 14/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 38ms/step - accuracy: 0.8178 - loss: 1.2693 - val_a
ccuracy: 0.7624 - val_loss: 1.4780
Epoch 15/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 35ms/step - accuracy: 0.8248 - loss: 1.2480 - val_a
ccuracy: 0.7724 - val_loss: 1.4601
Epoch 16/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 11s 38ms/step - accuracy: 0.8200 - loss: 1.2441 - val_a
ccuracy: 0.7741 - val_loss: 1.4640
Epoch 17/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 37ms/step - accuracy: 0.8296 - loss: 1.2014 - val_a
ccuracy: 0.7618 - val_loss: 1.4623
Epoch 18/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 9s 38ms/step - accuracy: 0.8313 - loss: 1.1835 - val_ac
curacy: 0.7730 - val_loss: 1.4487
Epoch 19/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 37ms/step - accuracy: 0.8374 - loss: 1.1541 - val_ac
```

```
curacy: 0.7724 - val_loss: 1.4469
Epoch 20/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 37ms/step - accuracy: 0.8452 - loss: 1.1340 - val_ac
curacy: 0.7774 - val_loss: 1.4158
```

In [35]:

```
import matplotlib.pyplot as plt
# Muestro gráfica de accuracy y losses
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 20), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 20), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 20), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 20), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
```

Out[35]:

```
<matplotlib.legend.Legend at 0x1e0000afe50>
```

In [36]:

```
# Evaluando el modelo de predicción con las imágenes de test
print("[INFO]: Evaluando red neuronal...")
model_reg.predict(x_test)
loss, accuracy = model_reg.evaluate(x_test, y_test)
print('Loss {}, accuracy {}'.format(loss,accuracy))
```

```
[INFO]: Evaluando red neuronal...
71/71 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step
71/71 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.7658 - loss: 1.4503
Loss 1.4502757787704468, accuracy 0.7658059000968933
```

- Dropout

In [37]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
# Vamos a codificar la topología de nuestro MLP
model_drop = Sequential()
model_drop.add(Dense(128,activation='relu', input_shape=(WORD_LIMIT,)))
#model_drop.add(Dropout(0.5))
model_drop.add(Dropout(0.55))
model_drop.add(Dense(64,activation='relu'))
#model_drop.add(Dropout(0.5))
model_drop.add(Dropout(0.55))
model_drop.add(Dense(46,activation='softmax')) # Reparto de la unidad de probabilid
```

```
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\keras\src\layers\core\dens
e.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
When using Sequential models, prefer using an `Input(shape)` object as the first lay
er in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [38]:
```
# Ahora que tengo definida la arquitectura, la compilo
model_drop.compile(optimizer='adam',
               loss='categorical_crossentropy', # ideal para clasificacion multicla
               metrics=['accuracy'])
```

In [39]:
```
# A entrenar nuestra red neuronal sea dicho!
H = model_drop.fit(x_train,y_train,epochs=20,batch_size=32, validation_split=0.2)
```

```
Epoch 1/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 12s 34ms/step - accuracy: 0.5333 - loss: 2.0259 - val_a
ccuracy: 0.7023 - val_loss: 1.3428
Epoch 2/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 31ms/step - accuracy: 0.6838 - loss: 1.3525 - val_ac
curacy: 0.7390 - val_loss: 1.1720
Epoch 3/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.7340 - loss: 1.1250 - val_ac
curacy: 0.7674 - val_loss: 1.0711
Epoch 4/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - accuracy: 0.7666 - loss: 0.9556 - val_a
ccuracy: 0.7869 - val_loss: 1.0298
Epoch 5/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 30ms/step - accuracy: 0.7916 - loss: 0.8298 - val_ac
curacy: 0.7852 - val_loss: 1.0163
Epoch 6/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.8117 - loss: 0.7573 - val_ac
curacy: 0.7896 - val_loss: 1.0258
Epoch 7/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.8221 - loss: 0.6833 - val_ac
curacy: 0.7908 - val_loss: 1.0014
Epoch 8/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - accuracy: 0.8365 - loss: 0.6376 - val_a
ccuracy: 0.7919 - val_loss: 1.0428
Epoch 9/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 29ms/step - accuracy: 0.8466 - loss: 0.5845 - val_ac
curacy: 0.7997 - val_loss: 1.0327
Epoch 10/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 31ms/step - accuracy: 0.8475 - loss: 0.5657 - val_ac
curacy: 0.8041 - val_loss: 1.0708
Epoch 11/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - accuracy: 0.8649 - loss: 0.5243 - val_a
ccuracy: 0.8013 - val_loss: 1.0528
Epoch 12/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 33ms/step - accuracy: 0.8633 - loss: 0.5085 - val_ac
curacy: 0.8013 - val_loss: 1.0563
Epoch 13/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.8720 - loss: 0.4787 - val_ac
curacy: 0.8008 - val_loss: 1.1051
Epoch 14/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 11s 33ms/step - accuracy: 0.8782 - loss: 0.4578 - val_a
ccuracy: 0.8058 - val_loss: 1.0841
Epoch 15/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 30ms/step - accuracy: 0.8774 - loss: 0.4400 - val_ac
curacy: 0.8047 - val_loss: 1.1218
Epoch 16/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 11s 33ms/step - accuracy: 0.8824 - loss: 0.4300 - val_a
ccuracy: 0.8063 - val_loss: 1.1316
Epoch 17/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.8870 - loss: 0.4121 - val_a
ccuracy: 0.8130 - val_loss: 1.1291
Epoch 18/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 34ms/step - accuracy: 0.8887 - loss: 0.4140 - val_ac
curacy: 0.8063 - val_loss: 1.1767
Epoch 19/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.8935 - loss: 0.3839 - val_ac
```

```
curacy: 0.8102 - val_loss: 1.2151
Epoch 20/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 33ms/step - accuracy: 0.8984 - loss: 0.3643 - val_ac
curacy: 0.8086 - val_loss: 1.2101
```

In [40]:

```
import matplotlib.pyplot as plt
# Muestro gráfica de accuracy y losses
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 20), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 20), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 20), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 20), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()

# ¿Y si desconectamos mayor porcentaje de neuronas?
```

Out[40]:

```
<matplotlib.legend.Legend at 0x1dfff037e50>
```

In [41]:

```
# Evaluando el modelo de predicción con las imágenes de test
print("[INFO]: Evaluando red neuronal...")
model_drop.predict(x_test)
loss, accuracy = model_drop.evaluate(x_test, y_test)
print('Loss {}, accuracy {}'.format(loss,accuracy))
```

```
[INFO]: Evaluando red neuronal...
71/71 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step
71/71 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.8001 - loss: 1.3116
Loss 1.3116341829299927, accuracy 0.8000890612602234
```

- Batch Normalization

In [42]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, BatchNormalization
# Vamos a codificar la topología de nuestro MLP
model_bn = Sequential()
model_bn.add(Dense(128,activation='relu', input_shape=(WORD_LIMIT,)))
model_bn.add(BatchNormalization())
model_bn.add(Dropout(0.6))
model_bn.add(Dense(64,activation='relu'))
model_bn.add(BatchNormalization())
model_bn.add(Dropout(0.6))
model_bn.add(Dense(46,activation='softmax')) # Reparto de la unidad de probabilidad
```

```
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\keras\src\layers\core\dens
e.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
When using Sequential models, prefer using an `Input(shape)` object as the first lay
er in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [43]:
```
# Ahora que tengo definida la arquitectura, la compilo
model_bn.compile(optimizer='adam',
               loss='categorical_crossentropy', # ideal para clasificacion multicla
               metrics=['accuracy'])
```

In [44]:
```
# A entrenar nuestra red neuronal sea dicho!
H = model_bn.fit(x_train,y_train,epochs=20,batch_size=32, validation_split=0.2)
```

```
Epoch 1/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 15s 35ms/step - accuracy: 0.4814 - loss: 2.5067 - val_a
ccuracy: 0.6834 - val_loss: 1.5868
Epoch 2/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.6601 - loss: 1.5596 - val_ac
curacy: 0.7490 - val_loss: 1.1993
Epoch 3/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 33ms/step - accuracy: 0.7133 - loss: 1.2653 - val_ac
curacy: 0.7668 - val_loss: 1.0947
Epoch 4/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 35ms/step - accuracy: 0.7474 - loss: 1.0817 - val_ac
curacy: 0.7796 - val_loss: 1.0397
Epoch 5/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.7680 - loss: 0.9768 - val_a
ccuracy: 0.7908 - val_loss: 0.9988
Epoch 6/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 34ms/step - accuracy: 0.7897 - loss: 0.8760 - val_ac
curacy: 0.7980 - val_loss: 0.9922
Epoch 7/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 33ms/step - accuracy: 0.8018 - loss: 0.8078 - val_ac
curacy: 0.8052 - val_loss: 0.9774
Epoch 8/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 34ms/step - accuracy: 0.8132 - loss: 0.7538 - val_ac
curacy: 0.8125 - val_loss: 0.9513
Epoch 9/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - accuracy: 0.8298 - loss: 0.6920 - val_a
ccuracy: 0.8036 - val_loss: 0.9415
Epoch 10/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 11s 33ms/step - accuracy: 0.8367 - loss: 0.6585 - val_a
ccuracy: 0.8086 - val_loss: 0.9584
Epoch 11/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 33ms/step - accuracy: 0.8469 - loss: 0.6007 - val_a
ccuracy: 0.8108 - val_loss: 0.9672
Epoch 12/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 33ms/step - accuracy: 0.8546 - loss: 0.5617 - val_ac
curacy: 0.8114 - val_loss: 0.9756
Epoch 13/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.8610 - loss: 0.5540 - val_ac
curacy: 0.8091 - val_loss: 0.9807
Epoch 14/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 34ms/step - accuracy: 0.8646 - loss: 0.5217 - val_ac
curacy: 0.8119 - val_loss: 0.9789
Epoch 15/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.8658 - loss: 0.5110 - val_ac
curacy: 0.8236 - val_loss: 0.9763
Epoch 16/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 35ms/step - accuracy: 0.8760 - loss: 0.4897 - val_ac
curacy: 0.8141 - val_loss: 1.0153
Epoch 17/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 36ms/step - accuracy: 0.8789 - loss: 0.4731 - val_ac
curacy: 0.8114 - val_loss: 1.0054
Epoch 18/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 37ms/step - accuracy: 0.8849 - loss: 0.4648 - val_ac
curacy: 0.8080 - val_loss: 1.0281
Epoch 19/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 10s 36ms/step - accuracy: 0.8853 - loss: 0.4452 - val_a
```

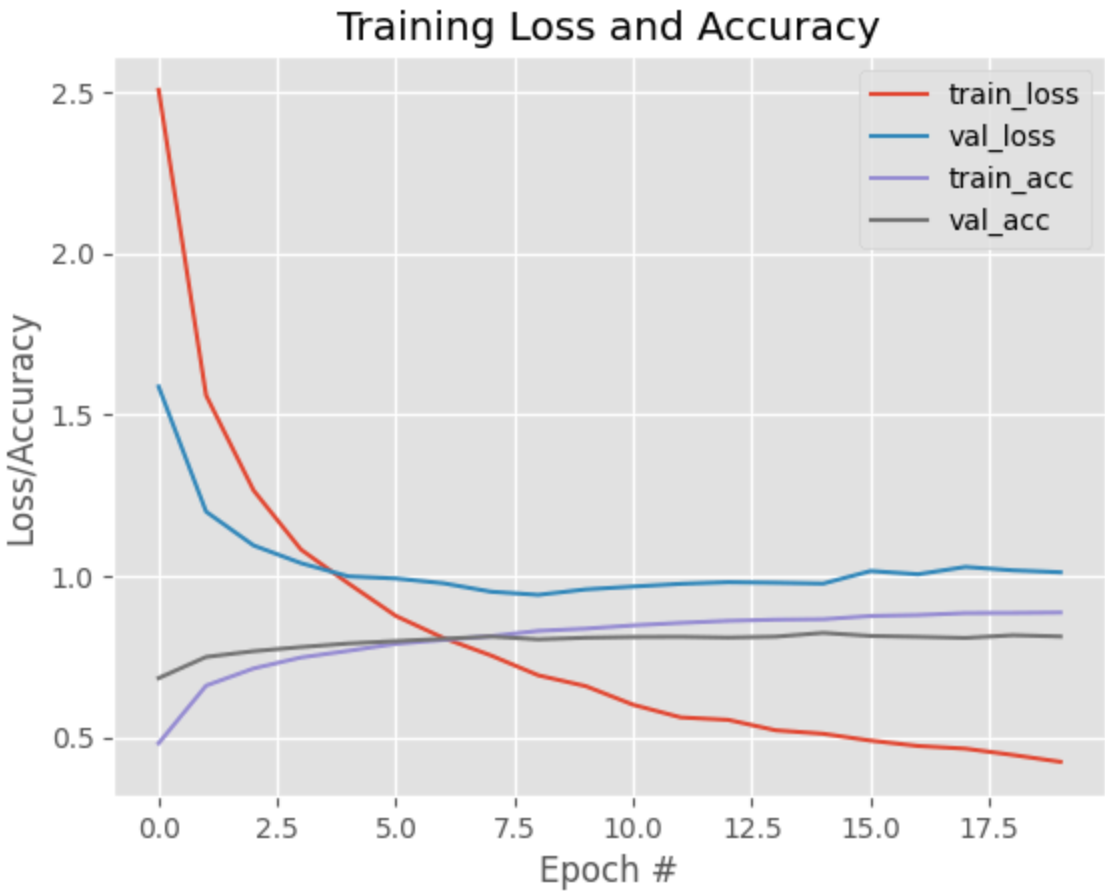
```
ccuracy: 0.8164 - val_loss: 1.0175
Epoch 20/20
225/225 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - accuracy: 0.8867 - loss: 0.4239 - val_ac
curacy: 0.8125 - val_loss: 1.0115
```

In [45]:

```python
import matplotlib.pyplot as plt
# Muestro gráfica de accuracy y losses
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 20), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 20), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 20), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 20), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
```

Out[45]:

```
<matplotlib.legend.Legend at 0x1dfff4c7190>
```



In [46]:

```python
# Evaluando el modelo de predicción con las imágenes de test
print("[INFO]: Evaluando red neuronal...")
model_bn.predict(x_test)
loss, accuracy = model_bn.evaluate(x_test, y_test)
print('Loss {}, accuracy {}'.format(loss,accuracy))
```

```
[INFO]: Evaluando red neuronal...
71/71 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step
71/71 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.7983 - loss: 1.0849
Loss 1.0848753452301025, accuracy 0.7983080744743347
```

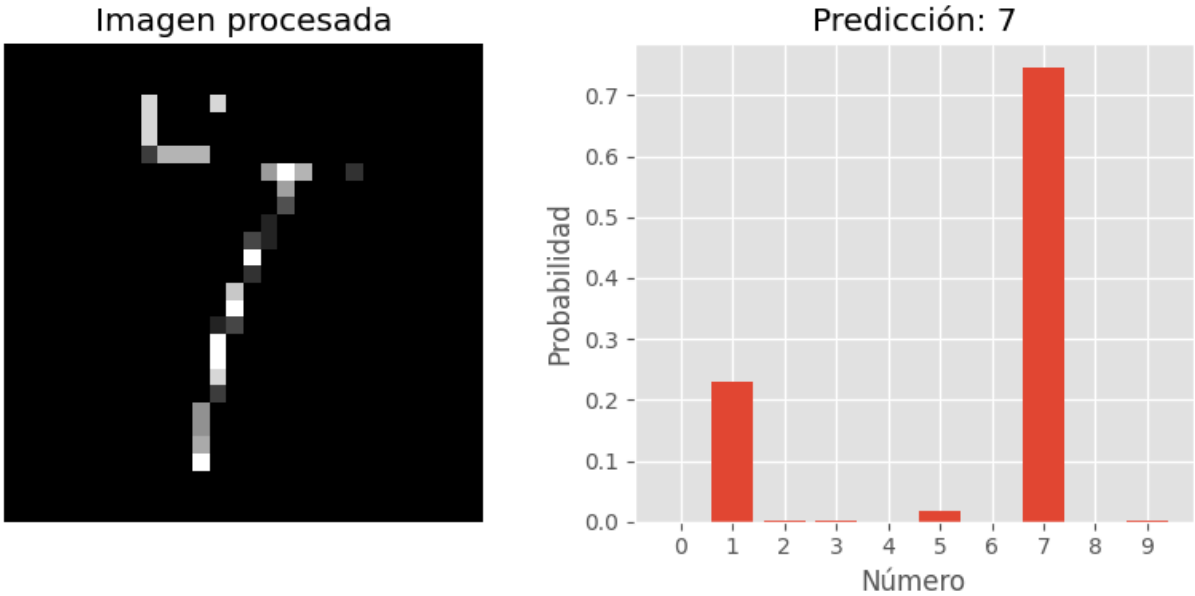
In [49]:

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Flatten, Dense, MaxPooling2D, Dropout,
from tensorflow.keras.callbacks import EarlyStopping

# ==================================================
# 1. Cargar datos de entrenamiento
# ==================================================
(x_train, y_train), _ = mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_train = x_train.reshape(-1, 28, 28, 1)

# ==================================================
# 2. Crear el modelo
# ==================================================
model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(32, (3, 3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D(2, 2),

    Conv2D(64, (3, 3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D(2, 2),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(10, activation='softmax')
])

# ==================================================
# 3. Compilar
# ==================================================
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# ==================================================
# 4. Entrenar
# ==================================================
parada = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
```

```python
)

model.fit(
    x_train, y_train,
    epochs=8,
    validation_split=0.1,
    callbacks=[parada],
    verbose=1
)

# ==================================================
# 5. Preparar imagen
# ==================================================
def preparar_imagen(ruta):
    imagen = cv2.imread(ruta, cv2.IMREAD_GRAYSCALE)

    if imagen is None:
        raise FileNotFoundError(f"No se pudo abrir la imagen: {ruta}")

    # Redimensionar para trabajar mejor
    imagen = cv2.resize(imagen, (100, 100))

    # Invertir colores: fondo negro, número blanco
    imagen = 255 - imagen

    # Pasar a blanco y negro
    _, imagen = cv2.threshold(imagen, 100, 255, cv2.THRESH_BINARY)

    # Buscar el número dentro de la imagen
    puntos = cv2.findNonZero(imagen)
    if puntos is None:
        raise ValueError("No se detectó ningún número en la imagen.")

    x, y, w, h = cv2.boundingRect(puntos)
    recorte = imagen[y:y+h, x:x+w]

    # Crear fondo cuadrado para centrar el número
    lado = max(w, h) + 20
    fondo = np.zeros((lado, lado), dtype=np.uint8)

    pos_x = (lado - w) // 2
    pos_y = (lado - h) // 2
    fondo[pos_y:pos_y+h, pos_x:pos_x+w] = recorte

    # Ajustar al tamaño de MNIST
    fondo = cv2.resize(fondo, (28, 28))
    fondo = fondo.astype("float32") / 255.0

    return fondo

# ==================================================
# 6. Hacer predicción
# ==================================================
ruta_imagen = "img/numeros.jpg"   # cambia esta ruta por tu imagen

try:
```

```
    imagen_lista = preparar_imagen(ruta_imagen)

    entrada = imagen_lista.reshape(1, 28, 28, 1)
    prediccion = model.predict(entrada, verbose=0)

    numero = np.argmax(prediccion)
    confianza = np.max(prediccion) * 100

    # Mostrar imagen y resultado
    plt.figure(figsize=(8, 4))

    plt.subplot(1, 2, 1)
    plt.imshow(imagen_lista, cmap="gray")
    plt.title("Imagen procesada")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.bar(range(10), prediccion[0])
    plt.xticks(range(10))
    plt.xlabel("Número")
    plt.ylabel("Probabilidad")
    plt.title(f"Predicción: {numero}")

    plt.tight_layout()
    plt.show()

    print(f"Número predicho: {numero}")
    print(f"Confianza: {confianza:.2f}%")

except Exception as e:
    print("Error:", e)
```

```
Epoch 1/8
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 52s 28ms/step - accuracy: 0.9524 - loss: 0.1570 - val
_accuracy: 0.9818 - val_loss: 0.0626
Epoch 2/8
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 47s 28ms/step - accuracy: 0.9791 - loss: 0.0729 - val
_accuracy: 0.9868 - val_loss: 0.0487
Epoch 3/8
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 78s 26ms/step - accuracy: 0.9837 - loss: 0.0552 - val
_accuracy: 0.9888 - val_loss: 0.0386
Epoch 4/8
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 86s 28ms/step - accuracy: 0.9877 - loss: 0.0418 - val
_accuracy: 0.9908 - val_loss: 0.0421
Epoch 5/8
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 46s 27ms/step - accuracy: 0.9887 - loss: 0.0388 - val
_accuracy: 0.9905 - val_loss: 0.0426
Epoch 6/8
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 43s 26ms/step - accuracy: 0.9902 - loss: 0.0326 - val
_accuracy: 0.9915 - val_loss: 0.0367
Epoch 7/8
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 46s 27ms/step - accuracy: 0.9917 - loss: 0.0275 - val
_accuracy: 0.9920 - val_loss: 0.0395
Epoch 8/8
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 46s 27ms/step - accuracy: 0.9925 - loss: 0.0253 - val
_accuracy: 0.9922 - val_loss: 0.0384
```




```
Número predicho: 7
Confianza: 74.67%
```

4. Github

https://github.com/Erick-305/machine-learning.git